

Lab 6: Simple Linear Regression

Author: Date: 2026-05-04

Topic Outline:

- Simple Linear Regression:
 - Motivating examples: a well-suited and a poorly-suited dataset.
 - Fitting a model using `statsmodels`.
 - Interpreting model output: coefficients, R^2 , and hypothesis tests.
 - Visualising best fit lines, confidence intervals, and prediction intervals.
 - Making new predictions.
 - Residual analysis and model diagnostics.
-

Collaboration

You are encouraged to collaborate with other students in your labs, but you are expected to write up your own work for submission. Please do not copy and paste other people's solutions to problems as it is considered plagiarism and you will be penalized and reported. Should you choose to collaborate with others, please note their names here:

Your name:

Collaborators:

1.

Agent Usage

Additionally, you are permitted to use resources such as ChatGPT and Claude to help you with your lab assignments and to enhance your learning experience. Please make a note of any agents you have used in this submission here:

Agents:

1. Introduction

In Lab 5 we concluded our exploratory data analysis by identifying a strong positive linear relationship between `study_hours_per_week` and `final_score`. This naturally raises the question: can we **model** this relationship formally and use it to make **predictions**?

Simple Linear Regression (SLR) is a statistical method for modelling the linear relationship between a single predictor variable x and a continuous response variable y . The model is:

$$Y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad \varepsilon_i \stackrel{\text{iid}}{\sim} N(0, \sigma^2),$$

where:

- β_0 is the **intercept** — the expected value of Y when $x = 0$.
- β_1 is the **slope** — the expected change in Y for a one-unit increase in x .
- ε_i are independent **errors** capturing variability not explained by x .

Given data $(x_1, y_1), \dots, (x_n, y_n)$, we estimate β_0 and β_1 via **Ordinary Least Squares (OLS)**, which minimises the **Residual Sum of Squares**:

$$\text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \text{where } \hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i.$$

In this lab we will:

1. Introduce two datasets — one well-suited to SLR and one that is not.
2. Fit SLR models using `statsmodels` and interpret the output.
3. Visualise best fit lines, confidence intervals, and prediction intervals.
4. Make predictions for new observations.
5. Perform residual analysis to assess model adequacy.

For this lab we will require the following packages:

```
In [2]: # Packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
from statsmodels.graphics.gofplots import qqplot

# Seaborn style
sns.set_style("whitegrid")
sns.set_palette("Set2")
```

2. The Datasets

In this lab we shall consider two data sets. The first is our student performance data from last week which we have already inspected and is well-suited for SLR. The second is a sythetic data set with a curved relationship that SLR cannot adequately capture. We will compare the two side-by-side to make clear both the methodology and the limitations of the method.

2.1 Dataset 1: Student Performance

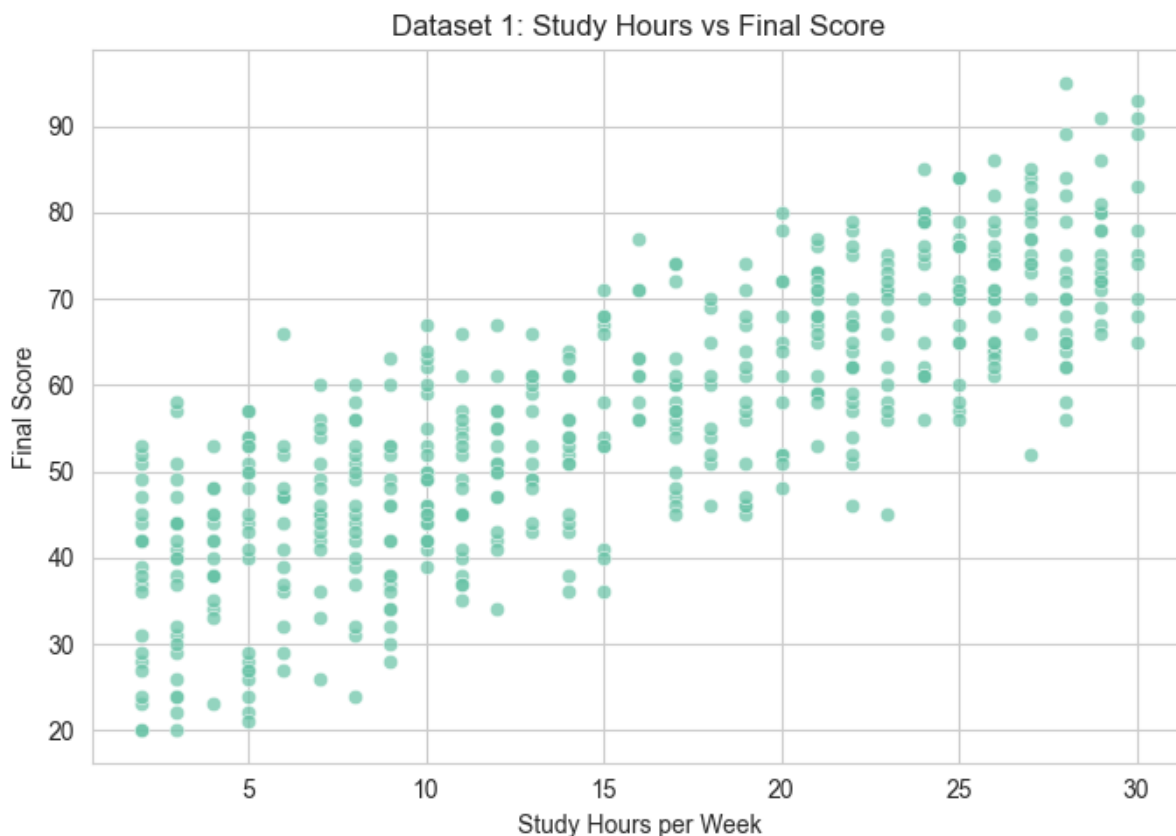
We load the `student_performance_cleaned.csv` data used in our previous 2 labs and isolate the predictor and response variables.

```
In [3]: # Generate Dataset 1: Advertising -> Sales (linear relationship)
df1 = pd.read_csv('student_performance.csv')
df1 = df1[['final_score', 'study_hours_per_week']]
df1.describe().round(2)
```

```
Out[3]:
```

	final_score	study_hours_per_week
count	500.00	500.00
mean	55.98	15.31
std	15.37	8.57
min	20.00	2.00
25%	45.00	8.00
50%	56.00	15.00
75%	68.00	23.00
max	95.00	30.00

```
In [4]: # Scatter plot of Dataset 1
plt.figure(figsize=(7, 5))
sns.scatterplot(data=df1, x='study_hours_per_week', y='final_score', a
plt.xlabel('Study Hours per Week')
plt.ylabel('Final Score')
plt.title('Dataset 1: Study Hours vs Final Score')
plt.tight_layout()
plt.show()
```



2.2 Dataset 2: Temperature and Daily Energy Consumption

Our second dataset records the mean daily outside temperature (°C) and the daily electricity consumption (MWh) for a large residential district across a year. Energy use tends to be **high in cold winters** (heating) and **high in hot summers** (air conditioning), producing a **U-shaped quadratic relationship** with temperature. This non-linearity makes Dataset 2 a poor candidate for SLR.

```
In [5]: # Generate Dataset 2: Temperature -> Energy consumption (curved relationship)
np.random.seed(7)
n2 = 200

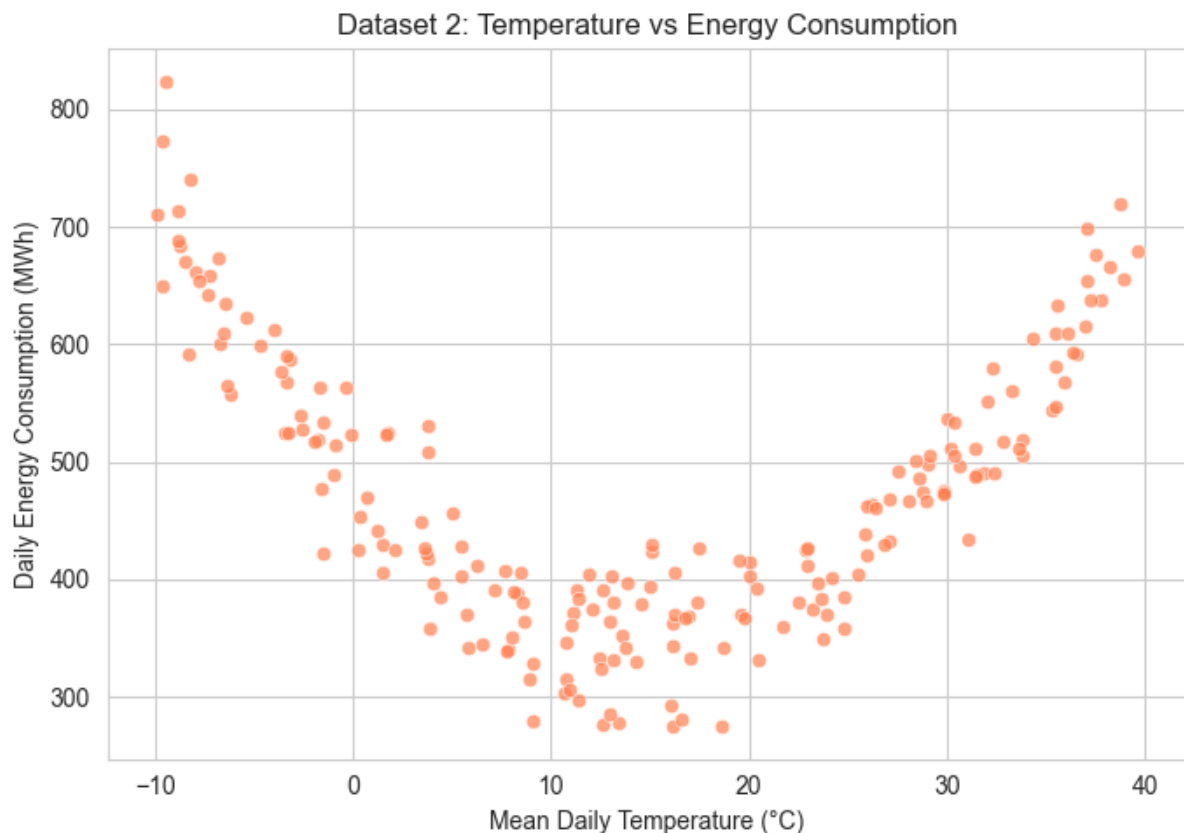
temperature = np.random.uniform(-10, 40, n2)
energy = 350 + 0.6 * (temperature - 15) ** 2 + np.random.normal(0, 40, n2)

df2 = pd.DataFrame({'temperature': temperature, 'energy': energy})
df2.describe().round(2)
```

Out [5]:

	temperature	energy
count	200.00	200.00
mean	14.47	470.00
std	14.24	117.08
min	-9.93	274.76
25%	3.09	380.40
50%	13.50	445.47
75%	27.07	548.67
max	39.63	823.74

```
In [6]: # Scatter plot of Dataset 2
plt.figure(figsize=(7, 5))
sns.scatterplot(data=df2, x='temperature', y='energy', alpha=0.7, color='orange')
plt.xlabel('Mean Daily Temperature (°C)')
plt.ylabel('Daily Energy Consumption (MWh)')
plt.title('Dataset 2: Temperature vs Energy Consumption')
plt.tight_layout()
plt.show()
```



Notice that Dataset 1 displays a clear positive linear trend, whereas Dataset 2 exhibits a pronounced U-shaped (quadratic) pattern. Keep this distinction in mind

as we proceed — the curvature in Dataset 2 will cause systematic problems for SLR.

3. Fitting Simple Linear Regression Models

We fit SLR models using the `statsmodels` library, which provides rich output including coefficient estimates, standard errors, p-values, and fit statistics. We use the **formula API** which allows us to specify models in a compact, readable way:

```
model = smf.ols('response ~ predictor', data=df).fit()
```

The string `'response ~ predictor'` is a **model formula**: it says "regress response on predictor". `statsmodels` automatically includes an intercept. Calling `.fit()` computes the OLS estimates $\hat{\beta}_0$ and $\hat{\beta}_1$.

We fit the model $\text{sales} = \beta_0 + \beta_1 \cdot \text{advertising} + \varepsilon$ and inspect the output:

```
In [7]: # Fit SLR model for Dataset 1
model1 = smf.ols('final_score ~ study_hours_per_week', data=df1).fit()
print(model1.summary())
```

OLS Regression Results

```

=====
=====
Dep. Variable:          final_score   R-squared:
0.646
Model:                  OLS          Adj. R-squared:
0.646
Method:                 Least Squares   F-statistic:
909.8
Date:                   Tue, 05 May 2026   Prob (F-statistic):      1.
86e-114
Time:                   14:26:46         Log-Likelihood:
-1815.5
No. Observations:      500             AIC:
3635.
Df Residuals:          498             BIC:
3643.
Df Model:               1
Covariance Type:       nonrobust
=====
=====
                                coef    std err          t      P>|t|      [
0.025    0.975]
-----
Intercept                33.8935    0.839    40.403    0.000    3
2.245    35.542
study_hours_per_week      1.4424    0.048    30.163    0.000
1.348    1.536
=====
=====
Omnibus:                 18.989   Durbin-Watson:
2.080
Prob(Omnibus):           0.000   Jarque-Bera (JB):
9.304
Skew:                    -0.095   Prob(JB):
0.00954
Kurtosis:                 2.359   Cond. No.
36.0
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

The key quantities to extract from the summary are:

Field	Location in output	Interpretation
coef — const	Coefficients table	$\hat{\beta}_0$: predicted sales when advertising = 0

coef — advertising	Coefficients table	$\hat{\beta}_1$: predicted change in sales per \$1000 increase in advertising
P>\ t\	Both rows	p-value for testing whether each coefficient equals zero
R-squared	Top block	Proportion of variance in sales explained by advertising
F-statistic	Top block	Overall test that the model explains significant variation

We see that our fitted model has estimates $\hat{\beta}_0 = 33.9$ - which is interpreted as the expected final score of a student who did not study for the course - and $\hat{\beta}_1 = 1.44$ - which gives the average score increase per hour per week studied. Both parameter estimates have a negligible p -value which indicates they are both highly significant. From $R^2 = 0.646$ we see that `study_hours_per_week` accounts for ~64% of the variance in `final_score`. Furthermore, the negligible F-statistic probability indicates highly significant evidence that $\beta_1 \neq 0$.

!! Problem

Q1 Fitting a Model

Fit a model to our second data set and produce our model summary. Discuss the parameter estimates, the p-values and the F test. Is this model a good fit?

Solution:

```
In [8]: model2 = smf.ols("energy ~ temperature", data = df2).fit()
print(model2.summary())
```


OLS Regression Results

```

=====
=====
Dep. Variable:          energy    R-squared:
0.005
Model:                  OLS      Adj. R-squared:
0.000
Method:                 Least Squares    F-statistic:
1.053
Date:                   Tue, 05 May 2026    Prob (F-statistic):
0.306
Time:                   14:26:50    Log-Likelihood:
-1235.3
No. Observations:      200    AIC:
2475.
Df Residuals:          198    BIC:
2481.
Df Model:               1
Covariance Type:       nonrobust
=====
=====

```

	coef	std err	t	P> t	[0.025
Intercept	478.6598	11.818	40.503	0.000	455.355
temperature	-0.5983	0.583	-1.026	0.306	-1.748

```

=====
=====
Omnibus:                11.347    Durbin-Watson:
1.992
Prob(Omnibus):          0.003    Jarque-Bera (JB):
10.267
Skew:                   0.486    Prob(JB):
0.00590
Kurtosis:               2.465    Cond. No.
29.0
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In []: The intercept $B_0 = 478.6598$ represents predicted energy consumption when temperature is 0°C. The slope $B_1 = -.5983$ explains that for every 1°C increase in temperature, energy consumption decreases by about 0.6 MWh. The intercept is significant. Temperature is not significant meaning that as temperature does not appear to be a significant predictor of energy consumption. F being .306 means it is not a good predictor.

Additionally the R^2 coef of determination being **0.05** shows this model

4. Best Fit Lines, Confidence Intervals, and Prediction Intervals

4.1 Confidence vs Prediction Intervals

After fitting, we can construct two types of interval at a new predictor value x^* :

95% Confidence Interval (CI) for the mean response — captures uncertainty about the true mean $E[Y | x^*] = \beta_0 + \beta_1 x^*$:

$$\hat{y}^* \pm t_{0.025, n-2} \cdot \hat{\sigma} \sqrt{\frac{1}{n} + \frac{(x^* - \bar{x})^2}{S_{xx}}}$$

95% Prediction Interval (PI) for a new individual observation — captures uncertainty about a single new Y at x^* :

$$\hat{y}^* \pm t_{0.025, n-2} \cdot \hat{\sigma} \sqrt{1 + \frac{1}{n} + \frac{(x^* - \bar{x})^2}{S_{xx}}}$$

The PI is always **wider** than the CI because it accounts for both uncertainty in the regression line *and* the natural variability of a single new observation (the extra 1 under the square root).

Both intervals **widen away from $\bar{x}$** — predictions near the centre of the data are more precise than those near the extremes, and extrapolating beyond the observed range becomes increasingly unreliable.

In `statsmodels`, both intervals are computed with a single call:

```
pred = model.get_prediction(new_x_dataframe)
pred_df = pred.summary_frame(alpha=0.05)
# Columns: mean, mean_ci_lower, mean_ci_upper (confidence
#          obs_ci_lower, obs_ci_upper          (prediction
#          interval)
```

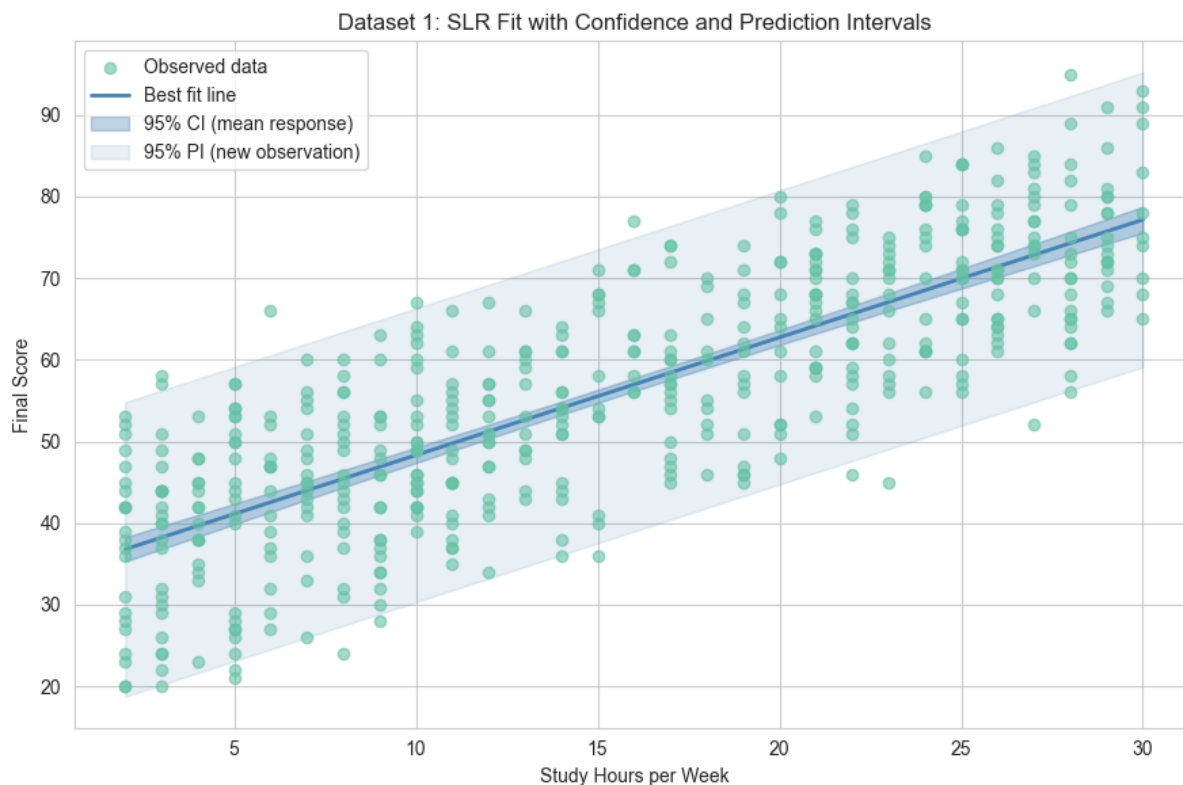
For our student performance data we first produce a dense grid of x values with which we produce our best fit line using the function `get_prediction`. From this we can reproduce our scatter plot with our best fit line and intervals overlayed. The intervals can be visualized as shaded regions with

`fill_between` which requires us to specify the upper and lower bound of the region.

```
In [9]: # Dense grid of x values for smooth curves
x_grid1 = pd.DataFrame(
    {'study_hours_per_week': np.linspace(
        df1['study_hours_per_week'].min(),
        df1['study_hours_per_week'].max(),
        300)
    })

# Compute fitted values and intervals across the grid
pred1 = model1.get_prediction(x_grid1).summary_frame(alpha=0.05)

# Plot
fig, ax = plt.subplots(figsize=(9, 6))
# Plot the points
ax.scatter(
    df1['study_hours_per_week'],
    df1['final_score'],
    alpha=0.6, zorder=3, label='Observed data'
)
# Plot the line
ax.plot(
    x_grid1['study_hours_per_week'],
    pred1['mean'],
    color='steelblue', lw=2, label='Best fit line')
# Plot the confidence and prediction intervals
ax.fill_between(
    x_grid1['study_hours_per_week'],
    pred1['mean_ci_lower'], pred1['mean_ci_upper'],
    alpha=0.35, color='steelblue', label='95% CI (mean response)'
)
ax.fill_between(
    x_grid1['study_hours_per_week'],
    pred1['obs_ci_lower'], pred1['obs_ci_upper'],
    alpha=0.12, color='steelblue', label='95% PI (new observation)'
)
ax.set_xlabel('Study Hours per Week')
ax.set_ylabel('Final Score')
ax.set_title('Dataset 1: SLR Fit with Confidence and Prediction Intervals')
ax.legend()
plt.tight_layout()
plt.show()
```



The best fit line closely tracks the data. The confidence interval (darker shaded band) is narrow, reflecting high precision in estimating the mean response. The prediction interval (lighter outer band) is wider, accommodating additional observation-level variability. Notice that both bands are narrowest near the centre of the data and flare outward at the extremes.

!! Problem

Q2 Best Fit Line and Intervals

Repeat this procedure for our synthetic data set. Comment on the differences you see in the width of the intervals and the path of the line through the points.

Solution:

```
In [10]: x_grid2 = pd.DataFrame(
    {'temperature': np.linspace(
        df2['temperature'].min(),
        df2['temperature'].max(),
        300)
    })

# Compute fitted values and intervals across the grid
pred2 = model2.get_prediction(x_grid2).summary_frame(alpha=0.05)
```

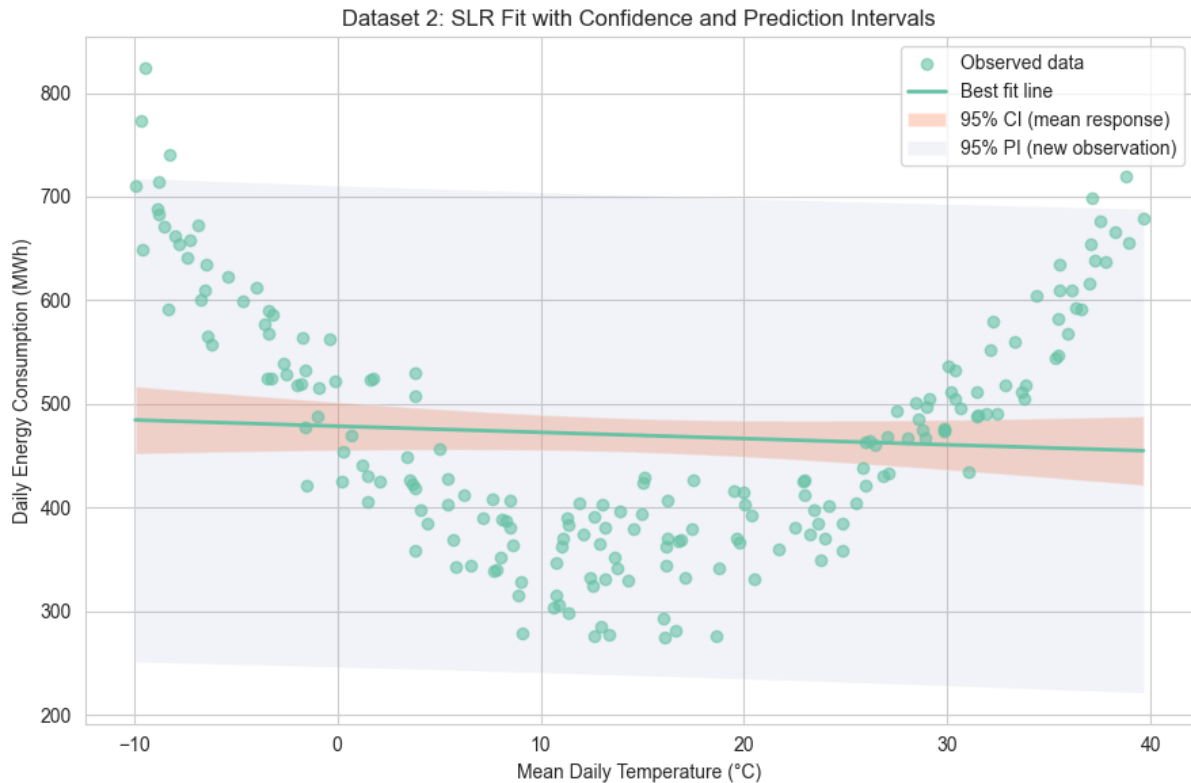
```
# Plot
fig, ax = plt.subplots(figsize=(9, 6))

# Plot the points
ax.scatter(
    df2['temperature'],
    df2['energy'],
    alpha=0.6, zorder=3, label='Observed data'
)

# Plot the line
ax.plot(
    x_grid2['temperature'],
    pred2['mean'],
    lw=2, label='Best fit line')

# Plot the confidence and prediction intervals
ax.fill_between(
    x_grid2['temperature'],
    pred2['mean_ci_lower'], pred2['mean_ci_upper'],
    alpha=0.35, label='95% CI (mean response)'
)
ax.fill_between(
    x_grid2['temperature'],
    pred2['obs_ci_lower'], pred2['obs_ci_upper'],
    alpha=0.12, label='95% PI (new observation)'
)

ax.set_xlabel('Mean Daily Temperature (°C)')
ax.set_ylabel('Daily Energy Consumption (MWh)')
ax.set_title('Dataset 2: SLR Fit with Confidence and Prediction Intervals')
ax.legend()
plt.tight_layout()
plt.show()
```



In []: The best fit line does **not** track the data. The confidence interval (da are poor at representing mean response **as** it does **not** follow the bell

5. Making New Predictions

Once the model is fitted we can predict the response at **new** predictor values. We pass a `DataFrame` with the same column name as the predictor used in the model formula. The `.get_prediction().summary_frame()` call returns both point predictions and interval predictions in a single table.

```
In [11]: # New student counts to predict test scores for
new_students = pd.DataFrame({'study_hours_per_week': [10, 15, 20, 25]})

# Full prediction summary: point estimate + CI + PI
pred_new1 = model1.get_prediction(new_students).summary_frame(alpha=0.
pred_new1.index = new_students['study_hours_per_week'].values
pred_new1.index.name = 'Study Hours per Week'

# Select and rename for readability
pred_new1 = pred_new1[['mean', 'mean_ci_lower', 'mean_ci_upper', 'obs_
pred_new1.columns = ['Final Score', 'CI Lower', 'CI Upper', 'PI Lower'
pred_new1.round(2)
```

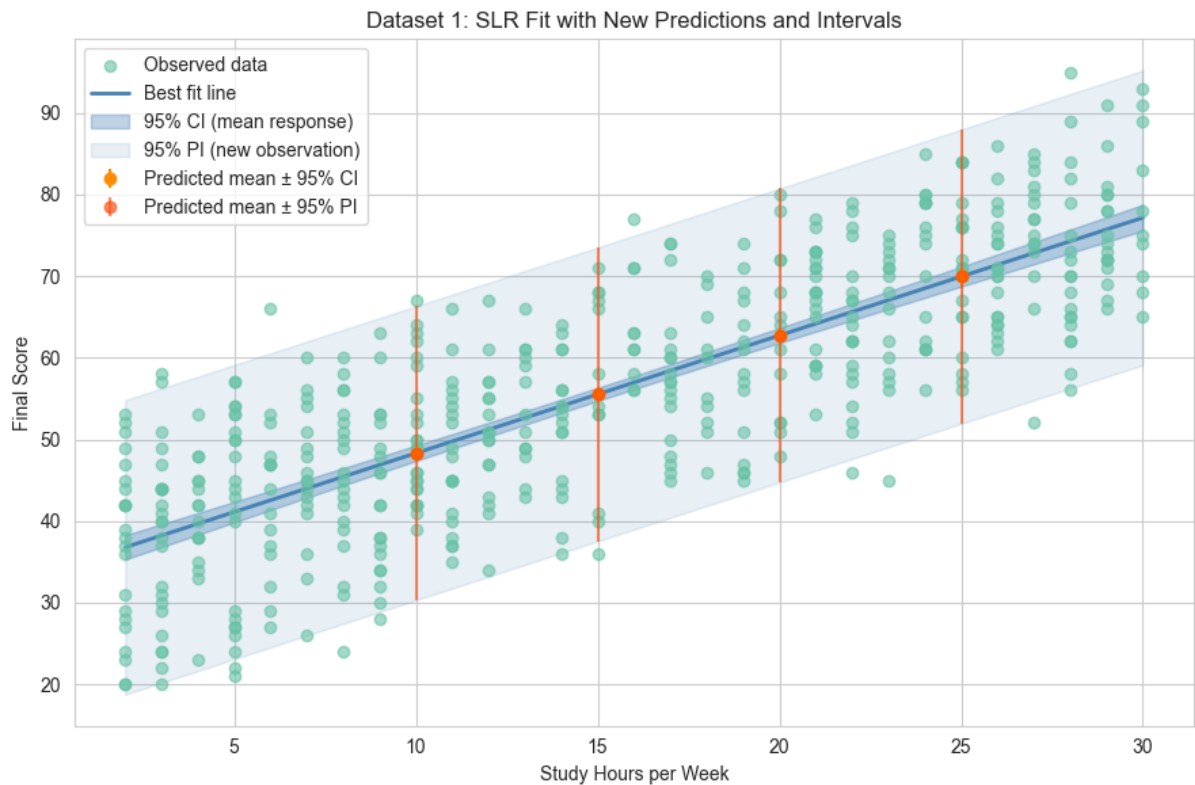
Out[11]:

	Final Score	CI Lower	CI Upper	PI Lower	PI Upper
Study Hours per Week					
10	48.32	47.37	49.26	30.31	66.33
15	55.53	54.73	56.33	37.53	73.53
20	62.74	61.83	63.66	44.74	80.75
25	69.95	68.74	71.17	51.93	87.98

```

In [12]: # Show new prediction on the scatter plot
fig, ax = plt.subplots(figsize=(9, 6))
ax.scatter(df1['study_hours_per_week'], df1['final_score'], alpha=0.6,
ax.plot(x_grid1['study_hours_per_week'], pred1['mean'], color='steelblue')
ax.fill_between(
    x_grid1['study_hours_per_week'],
    pred1['mean_ci_lower'], pred1['mean_ci_upper'],
    alpha=0.35, color='steelblue', label='95% CI (mean response)'
)
ax.fill_between(
    x_grid1['study_hours_per_week'],
    pred1['obs_ci_lower'], pred1['obs_ci_upper'],
    alpha=0.12, color='steelblue', label='95% PI (new observation)'
)
# Plot the new predictions with error bars
ax.errorbar(
    pred_new1.index, pred_new1['Final Score'],
    yerr=[pred_new1['Final Score'] - pred_new1['CI Lower'], pred_new1['Final Score'] - pred_new1['CI Upper']],
    fmt='o', color='darkorange', label='Predicted mean ± 95% CI', zorder=2
)
ax.errorbar(
    pred_new1.index, pred_new1['Final Score'],
    yerr=[pred_new1['Final Score'] - pred_new1['PI Lower'], pred_new1['Final Score'] - pred_new1['PI Upper']],
    fmt='o', color='orangered', alpha=0.6, label='Predicted mean ± 95% PI (new observation)'
)
ax.set_xlabel('Study Hours per Week')
ax.set_ylabel('Final Score')
ax.set_title('Dataset 1: SLR Fit with New Predictions and Intervals')
ax.legend()
plt.tight_layout()
plt.show()

```



The table shows for each study hours:

- **Final Score:** the point estimate $\hat{y}^* = \hat{\beta}_0 + \hat{\beta}_1 x^*$.
- **CI Lower / CI Upper:** the 95% CI for the *mean* final score across all students with those study hours.
- **PI Lower / PI Upper:** the 95% PI for a *single new* students final score with those study hours.

Observe that the PI is always wider than the CI.

!! Problem

Q3 Predictions

Repeat this process to compute predictions for our synthetic data set. Comment on the accuracy of your predictions.

Solution:

```
In [13]: # New temperature values to predict energy consumption for
new_temps = pd.DataFrame({'temperature': [0, 15, 30]})

# Full prediction summary: point estimate + CI + PI using the second model
pred_new2 = model2.get_prediction(new_temps).summary_frame(alpha=0.05)
```



```

pred_new2.index = new_temps['temperature'].values
pred_new2.index.name = 'Mean Daily Temperature (°C)'

# Select and rename for readability in the context of energy consumption
pred_new2 = pred_new2[['mean', 'mean_ci_lower', 'mean_ci_upper', 'obs_
pred_new2.columns = ['Energy Consumption', 'CI Lower', 'CI Upper', 'PI
pred_new2.round(2)

```

Out[13]:

	Energy Consumption	CI Lower	CI Upper	PI Lower	PI Upper
Mean Daily Temperature (°C)					
0	478.66	455.35	501.97	246.63	710.69
15	469.69	453.35	486.02	238.25	701.12
30	460.71	436.52	484.90	228.59	692.83

```

In [14]: fig, ax = plt.subplots(figsize=(9, 6))
ax.scatter(df2['temperature'], df2['energy'], alpha=0.6, zorder=3, col

# Plot the best fit line
ax.plot(x_grid2['temperature'], pred2['mean'], color='firebrick', lw=2

# Confidence Interval
ax.fill_between(
    x_grid2['temperature'],
    pred2['mean_ci_lower'], pred2['mean_ci_upper'],
    alpha=0.35, color='firebrick', label='95% CI (mean response)'
)

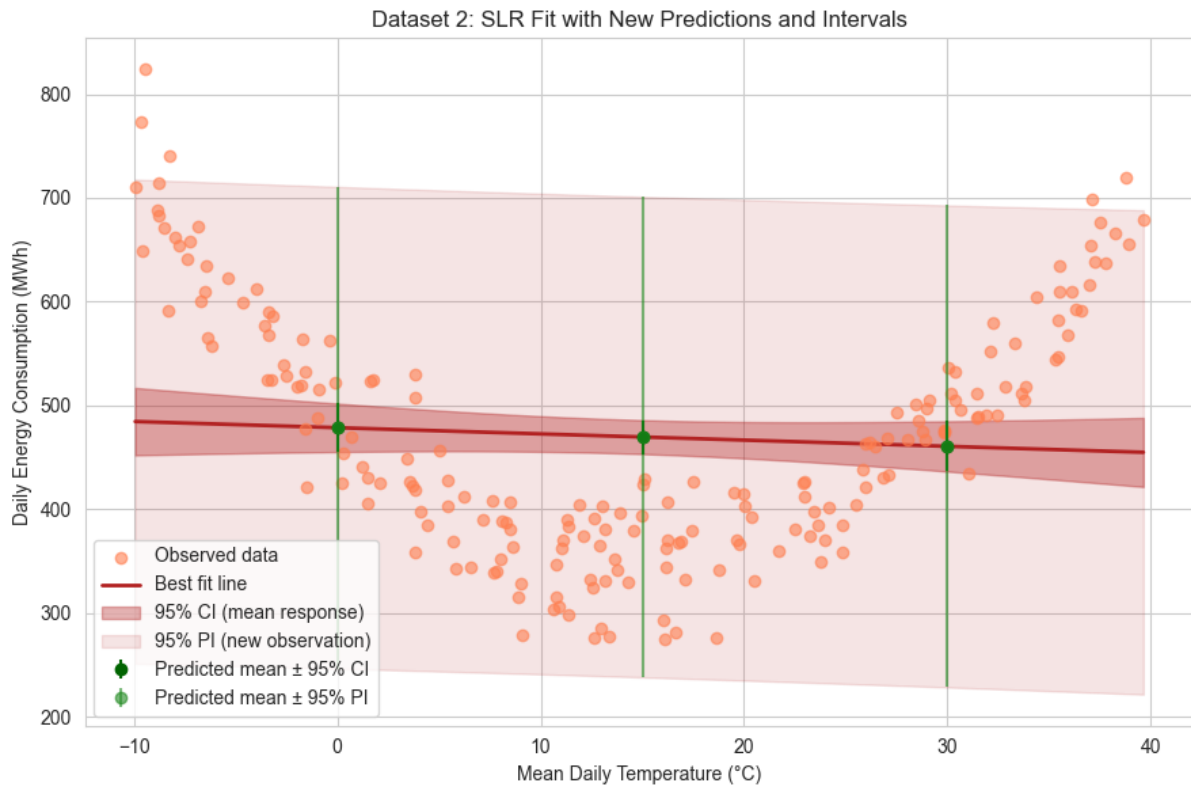
# Prediction Interval
ax.fill_between(
    x_grid2['temperature'],
    pred2['obs_ci_lower'], pred2['obs_ci_upper'],
    alpha=0.12, color='firebrick', label='95% PI (new observation)'
)

# Plot the new predictions with error bars
ax.errorbar(
    pred_new2.index, pred_new2['Energy Consumption'],
    yerr=[pred_new2['Energy Consumption'] - pred_new2['CI Lower'], pre
    fmt='o', color='darkgreen', label='Predicted mean ± 95% CI', zorde
)
ax.errorbar(
    pred_new2.index, pred_new2['Energy Consumption'],
    yerr=[pred_new2['Energy Consumption'] - pred_new2['PI Lower'], pre
    fmt='o', color='forestgreen', alpha=0.6, label='Predicted mean ± 9
)

ax.set_xlabel('Mean Daily Temperature (°C)')

```

```
ax.set_ylabel('Daily Energy Consumption (MWh)')
ax.set_title('Dataset 2: SLR Fit with New Predictions and Intervals')
ax.legend()
plt.tight_layout()
plt.show()
```



In []: This graph **is** still **not** very accurate. I mean the band **is** now wider wh
poor job at representing the graph

6. Residual Analysis

Fitting a model is only the first step — we must also **validate** that the model assumptions are satisfied. The key assumptions of SLR are:

1. **Linearity**: the true relationship between x and y is linear.
2. **Independence**: the errors ε_i are independent of each other.
3. **Homoscedasticity**: the errors have constant variance, $\text{Var}(\varepsilon_i) = \sigma^2$ for all i .
4. **Normality**: the errors follow a normal distribution.

We diagnose these assumptions by examining the **residuals** $e_i = y_i - \hat{y}_i$, which serve as estimates of the unknown errors ε_i . The two standard diagnostic plots are:

Residuals vs Fitted plot: plots e_i against $\hat{y}_i$. A well-fitting model shows residuals scattered randomly around zero with no pattern. A U-shaped or inverted-U pattern indicates non-linearity; a funnel shape indicates non-constant variance (heteroscedasticity).

Normal Q-Q plot: plots the quantiles of the residuals against the theoretical quantiles of the standard normal distribution. If residuals are approximately normal the points should fall close to the diagonal reference line. Systematic deviations suggest skewness or heavy tails.

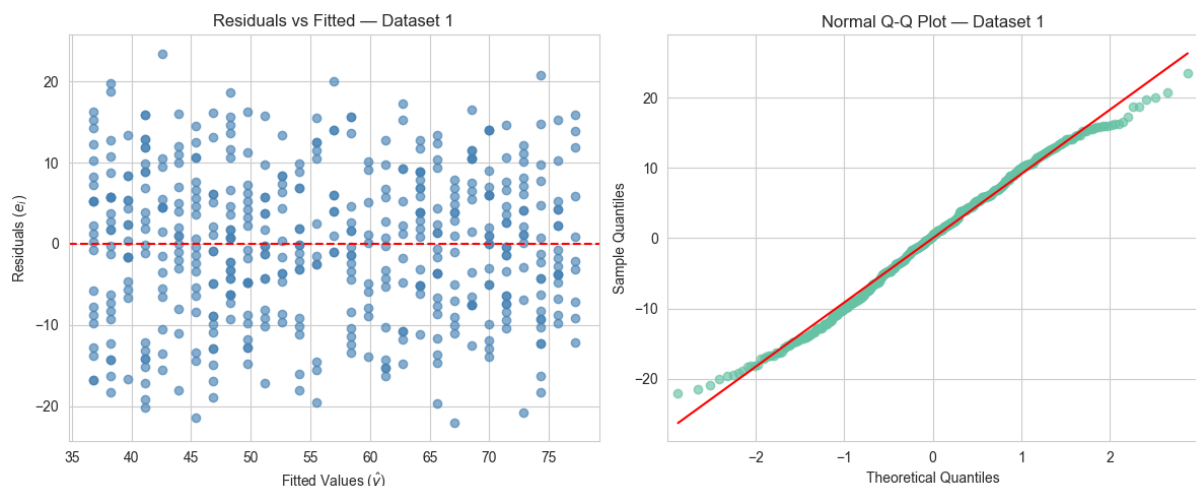
We produce these diagnostic plots for our student performance data and comment on our observations.

```
In [15]: fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Residuals vs Fitted
axes[0].scatter(model1.fittedvalues, model1.resid, alpha=0.65, color='red')
axes[0].axhline(0, color='red', linestyle='--', lw=1.5)
axes[0].set_xlabel('Fitted Values ( $\hat{y}$ )')
axes[0].set_ylabel('Residuals ( $e_i$ )')
axes[0].set_title('Residuals vs Fitted — Dataset 1')

# Normal Q-Q plot
qqplot(model1.resid, line='s', ax=axes[1], alpha=0.65)
axes[1].set_title('Normal Q-Q Plot — Dataset 1')

plt.tight_layout()
plt.show()
```



For Dataset 1:

- **Residuals vs Fitted:** the residuals appear to scatter randomly around zero with no discernible pattern — consistent with linearity and homoscedasticity.
- **Q-Q plot:** the points follow the diagonal closely — consistent with normality.

Both plots support the conclusion that SLR is an appropriate model for Dataset 1.

!! Problem

Q4 Residual Analysis

Reproduce these diagnostic plots for our synthetic data. What patterns do you see in the residual analysis? Are the residuals Gaussian?

Solution:

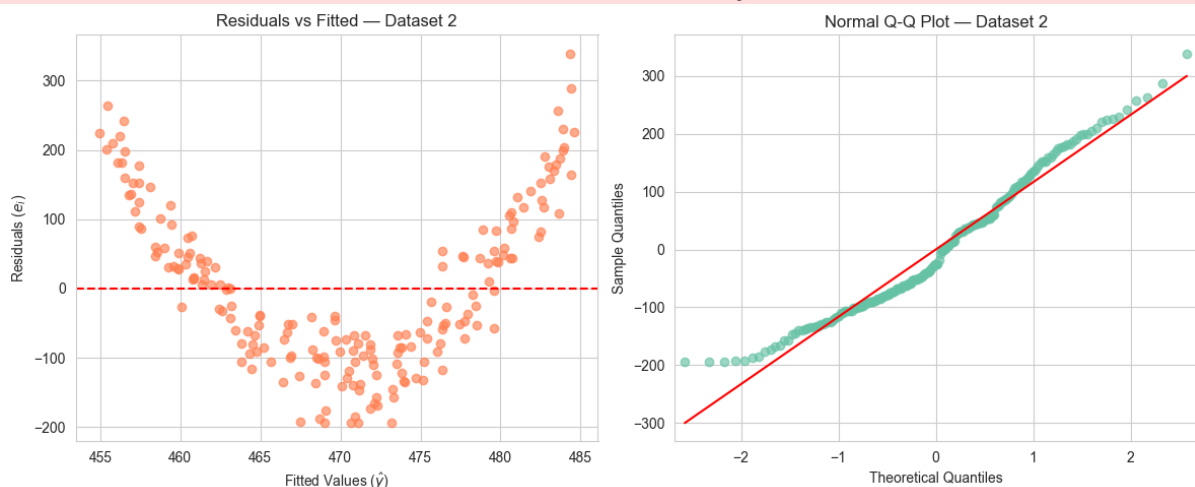
```
In [16]: fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Residuals vs Fitted
axes[0].scatter(model2.fittedvalues, model2.resid, alpha=0.65, color='red')
axes[0].axhline(0, color='red', linestyle='--', lw=1.5)
axes[0].set_xlabel('Fitted Values ( $\hat{y}$ )')
axes[0].set_ylabel('Residuals ( $e_i$ )')
axes[0].set_title('Residuals vs Fitted - Dataset 2')

# Normal Q-Q plot
qqplot(model2.resid, line='s', ax=axes[1], alpha=0.65)
axes[1].set_title('Normal Q-Q Plot - Dataset 2')

plt.tight_layout()
plt.show()
```

```
<>:6: SyntaxWarning: invalid escape sequence '\h'
<>:6: SyntaxWarning: invalid escape sequence '\h'
/var/folders/l6/dr99v_w91snds4bys0r_6qhw0000gn/T/ipykernel_20399/300348
0434.py:6: SyntaxWarning: invalid escape sequence '\h'
axes[0].set_xlabel('Fitted Values ( $\hat{y}$ )')
```



```
In [ ]: Residuals vs Fitted
very clear U-shaped pattern indicates that the relationship between te
```

non-linear and a straight line is failing to capture the trend.

Normal Q-Q Plot

The residuals deviate significantly from the diagonal line especially shape suggests that the residuals are not normally distributed

the SLR model is not appropriate. The residual analysis shows a violation of the normality assumption.

Submission Checklist

- Ensure you have completed all problems in the notebook.
- Clear your document code outputs and rerun the notebook from top to bottom, checking for any new errors.
- At the top of your notebook click more options -> export -> pdf.
 - Alternatively, export to html and save as a pdf.
- Make sure your pdf has all code chunks clearly visible and executed.
- Submit before the posted deadline.